Documentação de software · Clínica Veterinária

Modelo de treino. Aluno: _____ | Turma: _____ | Data: _____. O Anexo 1 não acompanha o PDF fornecido; transponha estas informações para o modelo indicado pelo avaliador.

Objetivo e usuários

Organizar cadastro de tutores e pets e agendamentos de uma clínica veterinária. Usuários de recepção e administração autenticados possuem acesso às funcionalidades; o Perfil identifica o usuário, mas esta versão não implementa permissões distintas por perfil.

Requisitos funcionais

Id Descrição

RF01 Autenticar por e-mail e senha; recusar credenciais inválidas com mensagem genérica.

RF02 Expirar a autenticação após 30 minutos e exigir novo login.

RF03 Exibir nome do usuário, links dos recursos e logout na Home.

RF04 Cadastrar, listar, editar e excluir tutores; impedir exclusão com dependências.

RF05 Buscar tutores por termo confirmado em nome, e-mail ou telefone.

RF06 Cifrar CPF antes de gravar e recuperá-lo somente em telas autenticadas.

RF07 Gerenciar pets com associação obrigatória a um tutor existente.

RF08 Gerenciar agendamentos, validando que o pet pertence ao tutor.

RF09 Listar agendamentos por data/hora crescente com todos os dados do tutor e pet.

RF10 Gerar script com estrutura e pelo menos três registros nas quatro tabelas de negócio.

Regras de negócio

Pet precisa de tutor existente. Agendamento exige pet pertencente ao tutor escolhido. Tutor/pet com dependências não podem ser removidos. Pet com agenda não pode trocar de tutor. Status aceitos: Agendado, Concluído e Cancelado. Listagem de agenda em ordem crescente de data/hora. CPF recebido com 11 dígitos sem pontuação (formato, sem validação dos dígitos verificadores). Campos obrigatórios e limites estão nos ViewModels e entidades.

Dados e arquitetura

ASP.NET Core MVC .NET 8, EF Core 8 e SQL Server. Fluxo: Razor · Controller · ViewModel/validação · serviços de JWT/AES · DbContext · SQL Server. Quatro tabelas de negócio: Usuarios, Tutores, Pets, Agendamentos. __EFMigrationsHistory é tabela técnica. Relacionamentos, atributos e chaves no diagrama anexo.

Autenticação e armazenamento

JWT HMAC-SHA256 com 30 minutos de validade, armazenado em cookie HttpOnly, Secure e SameSite Strict. Rotas protegidas por Authorize, POSTs com antiforgery. Senhas armazenadas como hashes BCrypt; CPF como AES-GCM (nonce 12 bytes, tag 16 bytes, chave 32 bytes). Chaves fora do código, por User Secrets em Development ou configuração do ambiente. Preserve a chave AES ao transportar o SQL. Logout remove o cookie, sem revogação global do JWT.

Instalação e execução

Restaurar pacotes e ferramenta EF previamente. Configurar instância SQL e chaves. Executar dotnet ef database update; iniciar em HTTPS (https://localhost:7243). A carga inicial de treino insere três registros em cada tabela vazia. Conta de demonstração: admin@clinica.com / Admin@123. Exportar saep_db.sql com --exportar-sql depois de conferir pelo menos três registros por tabela. Instruções detalhadas acompanham o README do código.

Critérios de aceitação

Demonstrar login incorreto e correto, Home com nome/menu/logout, busca por João, CRUDs completos, pet sem tutor rejeitado, agenda com tutor divergente rejeitada, listagem ordenada com dados completos, expiração de autenticação e CPF cifrado/senha com hash no SQL. Aplicar saep_db.sql em banco novo e confirmar contagens e FKs.

Limites do laboratório

Não possui controle de colisão de horários, renovação de tokens, revogação global, permissões por perfil, busca por CPF ou política de CPF único. A prova fornecida não determina esses recursos. O ambiente e as evidências de execução devem ser preenchidos pelo aluno.

Evidências do aluno

Ambiente: _____ | Resultado dos testes: _____ | Nome dos arquivos entregues: _____. Anexe evidências conforme orientação do avaliador.